

Міністерство освіти і науки України
Національний університет "Запорізька політехніка"

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторних робіт
з дисципліни

**“Проектування, моделювання та забезпечення якості
програмного забезпечення систем”**

для студентів магістратури

121 «Інженерія програмного забезпечення»

2023

Методичні вказівки до лабораторних робіт з дисципліни «Проектування, моделювання та забезпечення якості програмного забезпечення систем» для студентів магістратури спеціальності 121 «Інженерія програмного забезпечення» освітньої програми «Інженерія програмного забезпечення» всіх форм навчання / Г. В. Табунщик–Запоріжжя: НУЗП, 2023. – 50 с.

Автори: Г. В. Табунщик, к.т.н., професор

Рецензент: І.Я. Зеленева, к.т.н., доцент каф
«комп'ютерних систем та мереж»

Відповідальний
за випуск: Г.В. Табунщик, к.т.н., професор

Затверджено
на засіданні кафедри
«Програмних засобів»
9 червня 2023 р.
Протокол № 12

ЗМІСТ

Вступ.....	4
Лабораторна робота №1 Розроблення веб-застосунків з використанням Django.....	5
Лабораторна робота №2 Використання AWS Green Grass	26
Лабораторна робота №3 Програмування для ривня пристроїв з використанням Python.....	30
Лабораторна робота №4 Принципи тестування у Django	37
Лабораторна робота № 5. Розроблення підсистем аналізу даних	43
Лабораторна робота №6. Взаємодія з базами даних	45

ВСТУП

Метою даного курсу є навчити студентів використовувати навички об'єктно-орієнтованого аналізу та проектування для проектування, моделювання програмного забезпечення систем з урахуванням вимог до якості, безпеки та надійності. при використанні мови проектування UML, мови програмування Python, навчити створювати веб-застосування та виконувати модульне тестування за допомогою фреймворку Django та інших сучасних засобів розробки програмного забезпечення у напрямку веб, на базі одноплатного комп'ютеру Raspberry Pi.

На основі вивчення дисципліни «Проектування, моделювання та забезпечення якості програмного забезпечення систем» студент отримає комплекс знань, необхідних для зменшення складності та забезпечення відповідного рівня якості програмного забезпечення систем; зможе ефективно використовувати сучасні технології для моделювання, проектування та розробки програмного забезпечення систем.

Після вивчення дисципліни «Проектування, моделювання та забезпечення якості програмного забезпечення систем» студент повинен знати етапи життєвого циклу програмного забезпечення, елементи об'єктної моделі, процедури об'єктно-орієнтованого аналізу, процедури об'єктно-орієнтованого проектування, основи уніфікованої мови моделювання UML, основні концепції уніфікованого процесу розробки програмного забезпечення UP, архітектуру програмного забезпечення, принципи розробки та архітектуру Django, порядок створення веб-застосування у Django та налаштувань WSGI, відомості щодо модульного тестування та тестування у Django зокрема, принципи запуску сервера із Nginx, роботу зі хмарними базами даних.

Для виконання лабораторних робіт рекомендується використовувати PyCharm IDE як для консольних Python програм, так і для Django веб-застосувань, для діаграм UML рекомендовано використовувати вільне програмне забезпечення.

Матеріал підготовлено на основі матеріалів, створених під час виконання міжнародного проєкту VIMUK Німецької служби академічних обмінів DAAD, та на основі матеріалів Luxsoft LTD

ЛАБОРАТОРНА РОБОТА №1 РОЗРОБЛЕННЯ ВЕБ-ЗАСТОСУВАНЬ З ВИКОРИСТАННЯМ DJANGO

Мета роботи

Ознайомитися із основними можливостями фреймворку Django, навчитися створювати та запускати проекти із використанням утиліти Supervisor та Nginx сервера.

Теоретичні відомості

Django фреймворк. У якості основи для розроблення підсистеми можна використовувати Django – фреймворк (програмний каркас) з багатими можливостями, що підходить для розробки складних сайтів і веб-застосовувань, написаний мовою програмування Python.

Один із основних принципів фреймворку – DRY (don't repeat yourself). Дотримання принципу програмування DRY дозволяє домогтися високого рівня супроводжуваності проекту: простоти внесення змін і якісного тестування. Веб-системи на Django будуються з одного або декількох застосовувань, які рекомендується робити відокремленими і такими, що підключаються. Це одне з помітних архітектурних відмінностей цього фреймворку від деяких інших (наприклад, Ruby on Rails). Також, на відміну від багатьох інших фреймворків, обробники URL в Django конфігуруються у явному вигляді (за допомогою регулярних виразів), а не задаються автоматично зі структури контролерів.

Для роботи з базою даних Django використовує власний ORM (англ. Object-Relational Mapping, укр. об'єктно-реляційне відображення), в якому модель даних описується класами Python, і за ним генерується схема бази даних.

Архітектура Django схожа на «Модель-Подання-Контролер» (MVC). Контролер класичної моделі MVC приблизно відповідає рівню, який в Django називається Уявлення (View), а презентаційна логіка Подання реалізується в Django рівнем Шаблонів (Templates). Через цю багаторівневу архітектуру Django часто називають

архітектурою типу «Модель-Шаблон-Подання» (MTV). Загальна схема зображена на рис. 2.1.

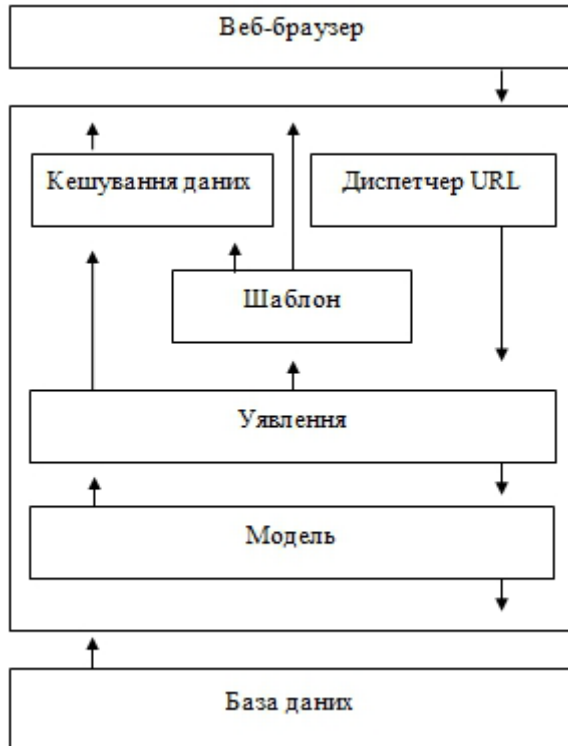


Рисунок 2.1 – Django MTV архітектура

Фреймворк надає ряд засобів, які допомагають у швидкій розробці веб-застосунків. Наприклад, не потрібно створювати контролери та сторінки для адміністративної частини сайту, в Django є вбудований застосунок для управління вмістом, який можна включити в будь-який застосунок, зроблений на Django, і який може керувати відразу декількома веб застосуваннями на одному сервері. Адміністративне застосування дозволяє створювати, змінювати і видаляти будь-які об'єкти наповнення веб застосунку, протоколюючи всі вчинені дії, і надає інтерфейс для управління користувачами і групами (з пооб'єктним призначенням прав).

Повна схема роботи Django-застосування наведена на рис. 2.2.

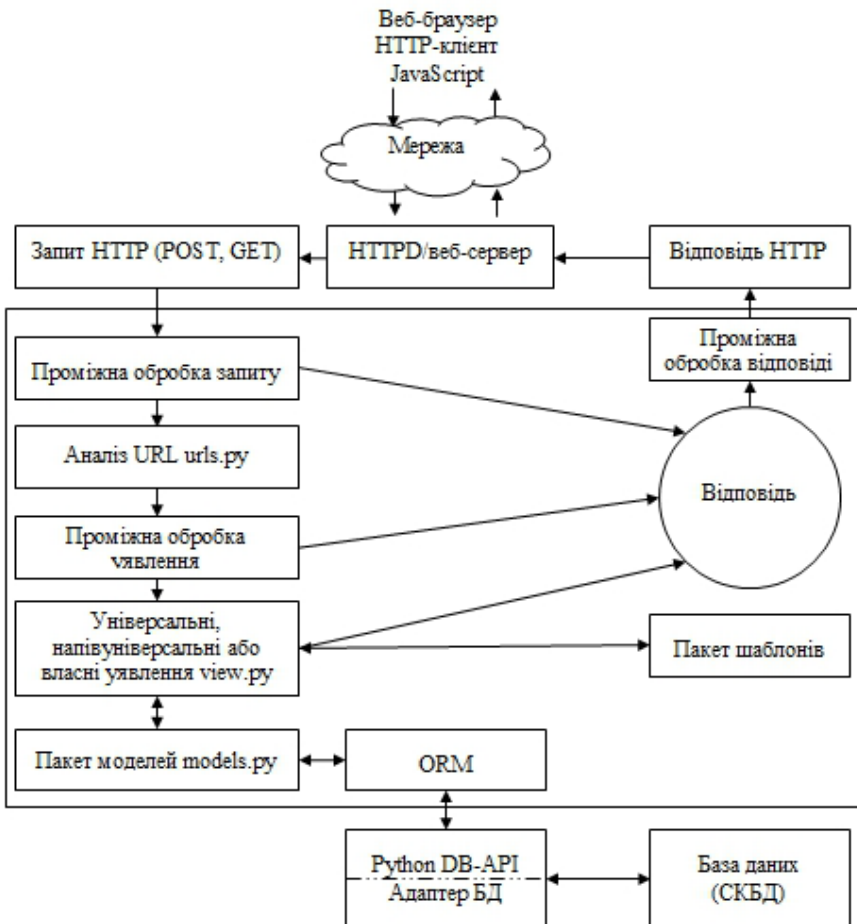


Рисунок 2.2 – Зображення роботи застосування

Віртуальне середовище. Віртуальне оточення для Python – дуже зручний інструмент при одночасній роботі з декількома проектами. При розробці встановлюються різні бібліотеки, версія самого Python також може відрізнятися.

Використання віртуального середовища дозволяє абстрагуватися від бібліотек, що використовуються в системі. Створюється віртуальне оточення, активується, і після цього всі модулі Python будуть встановлені лише в даному віртуальному оточенні. Щоб працювати в іншому проєкті з іншими версіями бібліотек, досить просто перемкнути віртуальне оточення.

`virtualenv` – власне утиліта для створення віртуальних оточень.

Для установки досить виконати:

```
sudo apt-get install python-pip
pip install virtualenv
```

Для створення віртуального оточення, виконайте:

```
virtualenv <ім'я віртуального оточення>
```

У поточному каталозі буде створена нова директорія з вказаною вами назвою, куди будуть перенесені `python`, `pip` і надалі встановлені інші бібліотеки.

Активація віртуального оточення проводиться командою:

```
<ім'я віртуального оточення>\bin\activate
```

Після активування, перед ім'ям користувача з'явиться назва віртуального середовища у дужках, наприклад:

```
(env)user@host:~/django-project$
```

Для деактивації достатньо виконати:

```
deactivate
```

Перед встановленням будь-яких залежностей для проєкту (бібліотек) середовище повинне бути активоване.

Створення Django-проєкту. Встановимо залежність Django та створимо проєкт:

```
pip install Django
```



```
django-admin.py startproject.djangosite
```

Після виконання останньої програми з'явиться нова папка з відповідним ім'ям:

```
cd.djangosite/
```

Структура проекту виглядає наступним чином:

```

.
├──.djangosite
│   ├──__init__.py
│   ├──settings.py
│   ├──urls.py
│   └──wsgi.py
└──manage.py

```

- .djangosite /__init__.py - порожній файл, який вказує Python, що поточний каталог є пакетом Python;
- .djangosite /settings.py - налаштування/конфігурація проекту;
- .djangosite /urls.py - конфігурація URL для проекту Django;
- .djangosite /wsgi.py - точка входу проекту для WSGI-сумісних веб-серверів;
- ./manage.py - скрипт для взаємодії із проектом Django.

Розглянемо детальніше файл з налаштуваннями. Змінна `INSTALLED_APPS` містить додатки Django, які використовуються в проєкті. За замовчуванням, вона містить наступні програми, всі вони включені в Django:

- django.contrib.admin – інтерфейс адміністратора;
- django.contrib.auth – система аутентифікації;
- django.contrib.contenttypes – фреймворк, що надає високорівневий, узагальнений інтерфейс для роботи з моделями;
- django.contrib.sessions – фреймворк сесії;
- django.contrib.messages – фреймворк повідомлень;
- django.contrib.staticfiles – фреймворк для роботи зі статичними файлами.

Ці програми включаються за замовчуванням. Деякі з цих програм можуть використовувати принаймні одну таблицю в базі

даних, тому необхідно створити базу даних. Для цього виконайте наступну команду:

```
python manage.py migrate
```

За замовчуванням використовується SQLite база даних. SQLite включений в Python, тому не потрібно встановлювати жодних додаткових бібліотек.

Команда `migrate` аналізує значення `INSTALLED_APPS` і створює всі необхідні таблиці в базі даних, використовуючи налаштування бази даних з файлу `settings.py` та міграції з застосування (про це далі).

Для того, щоб перевірити правильність первинних налаштувань, необхідно перейти до каталогу із файлом `manage.py` та виконати команду:

```
python manage.py runserver
```

У терміналі буде наступне:

```
Performing system checks...
System check identified no issues (0 silenced).
October 08, 2016 - 20:15:07
Django version 1.10.2, using settings
'djangosite.settings'
Starting development server at
http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Якщо перейти за посиланням `http://127.0.0.1:8000/` можна побачити стандартну сторінку Django (рис. 2.3).

It worked!

Congratulations on your first Django-powered page.

Of course, you haven't actually done any work yet. Next, start your first app by running `python manage.py startapp [app_label]`.

You're seeing this message because you have `DEBUG = True` in your Django settings file and you haven't configured any URLs. Get to work!

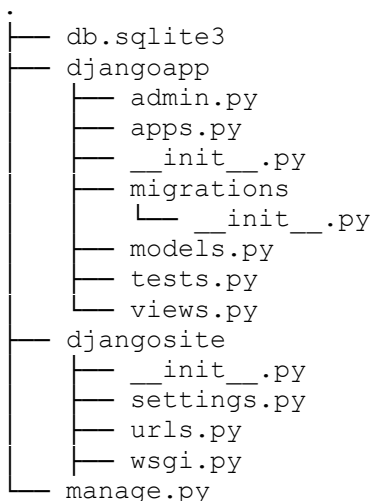
Рисунок 2.3 – Сторінка тестового запуску Django

Створення застосування. Кожний проект Django складається з пакетів Python (застосування). Застосування надає певний функціонал – наприклад, web-блог. Проект – це сукупність програм і конфігурації сайту. Проект може містити кілька застосувань.

Для створення застосування треба виконати:

```
python manage.py startapp djangoapp
```

`djangoapp` — назва застосування. Після цього змінилась структура проекту:



Про структуру застосування:

- `djangoapp /admin.py` — файл для налаштування адмінської частини;
- `djangoapp/apps.py` — файл керування налаштуваннями застосування;
- `djangoapp/migrations` — каталог, що містить файли міграцій;
- `djangoapp/models.py` — файл моделей проекту, що описують структуру бази даних;
- `djangoapp /tests.py` — файл для тестування;
- `djangoapp/views.py` — файли, у яких описується логіка

програми та обробка запитів за URL.

Створене застосування необхідно додати до змінної `INSTALLED_APPS` у файлі налаштувань `django/settings.py`:

```
INSTALLED_APPS = [
    ...,
    'djangoapp'
]
```

Створення моделей та бази даних. Відредагуйте файл `djangoapp/models.py`, щоб він виглядав наступним чином:

```
from __future__ import unicode_literals
from django.db import models

class Measurement(models.Model):
    value = models.CharField(max_length=50)
    description = models.CharField(max_length=250,
    blank=True, null=True)

    def __unicode__(self):
        return self.value
```

Модель представлена класом, успадкованим від `django.db.models.Model`. Вона містить декілька атрибутів: значення та опис, кожен з яких відображає поле в таблиці бази даних. Опис є необов'язковим полем. Кожне поле є екземпляром класу `Field` – наприклад, `CharField` для текстових полів. Це вказує Django які типи даних зберігають ці поля. Метод `__unicode__` відповідає за репрезентацію об'єкту, повертає `str`.

Виконайте команду:

```
./manage.py makemigrations
```

Ви повинні побачити наступне:

```
Migrations for 'djangoapp':
  djangoapp/migrations/0001_initial.py:
    - Create model Measurement
```

Після цього:

```
./manage.py migrate
```

Після виконання `makemigrations`, Django відстежує зміни у ваших моделях та зберігає їх до міграцій. Міграції використовуються Django для збереження змін ваших моделей (і структури бази даних) - це просто файли на диску. Ви можете вивчити міграцію для створення ваших моделей, вона знаходиться у файлі `djangoapp/migrations/0001_initial.py`.

Команда `migrate` виконує міграції і автоматично оновлює базу даних.

HTML шаблони Django. Для використання шаблонів необхідно створити каталог `templates` в кореневому каталозі проекту поруч із `manage.py` та внести зміни до файлу налаштувань в змінній `TEMPLATES` за ключем `DIRS` додати, щоб це виглядало наступним чином:

```
TEMPLATES = [
    {
        ...,
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        ...
    },
]
```

Також додайте два шаблони до каталогу `templates`: `measurement_list.html` та `create_measurement.html`

View у Django. Уявлення – це "тип" сторінок вашого застосування, що є функцією для обробки запиту та використовує шаблон для генерації сторінок.

Для створення першого уявлення додамо текстовий файл `test_data`. Він повинен лежати поряд з `manage.py`. Нехай його зміст буде наступним:

```
7483728478329
```

```

7483784738924
8989438948394
4787438784758
8958439589493
8594389543905
5843958943895

```

Відредагуйте файл `djangoapp/views.py`:

```

import os
from django.views import generic
from django.conf import settings
from djangoapp.models import Measurement

class MeasurementList(generic.ListView):
    model = Measurement
    context_object_name = 'measurements'
    template_name = 'measurement_list.html'

    def get(self, request, *args, **kwargs):
        try:
            file_path =
os.path.join(settings.BASE_DIR, 'test_data')
            measurement_file = open(file_path, 'r')
            for value in measurement_file:

Measurement.objects.get_or_create(value=value)
            except IOError:
                pass

        return super(MeasurementList,
self).get(request, *args, **kwargs)

```

В Django є безліч класів, що спрощують процес розробки проєктів, наприклад, клас `ListView` призначений для виведення списків об'єктів. Він дозволяє отримати об'єкти, модель яких вказана у атрибуті `model` класу, та використовувати у HTML шаблоні за допомогою змінної у атрибуті `context_object_name`. `template_name` дозволяє вказати назву шаблону, що буде використовуватися.

Відредагуємо шаблон `measurement_list.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Виміри</title>
</head>
<body>
    <h3>Виміри</h3>
    {% for measurement in measurements %}
        <p>{{ measurement }}</p>
    {% endfor %}
</body>
</html>
```

В шаблоні використовується ітератор `for`, який перебирає масив з об'єктів `Measurement`. Доступ до змінної здійснюється за допомогою «`{{ }}`». Саме тут можна побачити застосування методу `__unicode__` об'єкту, адже об'єкт виглядає як рядок.

Цього було б достатньо для використання уявлення. Але перевизначивши метод `get`, ми додали більше функціональності до нашого уявлення. Відкривається файл `test_data` та зчитуються дані, при цьому вони зберігаються у базу даних. Метод `get_or_create` дозволяє створити об'єкт в базі даних, тільки якщо такого ще не існує, що дозволяє уникнути дублювання даних.

URL адреса. Для того щоб переглянути результат, додамо URL шаблон до `django.urls.py`:

```
from django.conf.urls import url
from django.contrib import admin
from djangoapp.views import MeasurementList
urlpatterns = [
    url(r'^admin/', admin.site.urls),
    url(r'^$', MeasurementList.as_view(),
name='measurement_list')
]
```

URL-шаблон - це загальна форма URL-а. Наприклад: `/measurement/<id>/`.

Щоб з URL-а отримати уявлення, Django використовується так званий 'URLconf'. URLconf визначає відповідність URL-шаблонів (є регулярними виразами) і уявлень.

Тепер якщо запустити сервер та перейти за адресою `http://127.0.0.1:8000/`, можна буде побачити список об'єктів Measurement за даними з файлу `test_data`, вони додані до бази даних. Спробуйте додати рядок із новими даними до файлу та оновити сторінку.

Форми. Якщо ви плануєте створювати сайти і застосування, що приймають і зберігають дані, вам необхідно використовувати форми.

Форма в HTML – це набір елементів `<form>...</form>`, які дозволяють користувачу вводити текст, вибирати опції, змінювати об'єкти сторінки, і так далі, а потім відправляти цю інформацію на сервер.

Крім `<input>` елементів форма повинна містити ще дві речі:

- куди: URL, на який будуть відправлені дані;
- як: HTTP метод, який повинна використовувати форма для відправки даних.

GET і POST – єдині HTTP методи, які використовуються для форм. Будь-який запит, який може змінити стан системи - наприклад, який змінює дані в базі даних - повинен використовувати POST. GET повинен використовуватися для запитів, які не впливають на стан системи. Не слід використовувати GET для форми з паролем, тому що пароль з'явиться в URL, а отже в історії браузера і логах сервера. Також він не підходить для відправки великої кількості даних або бінарних даних, наприклад, зображення. Web-застосування, яке використовує GET запити для аутентифікації можна буде легко зламати: не складно підробити форму для запиту на сервер і отримати важливі дані про систему. POST використовує додаткові механізми захисту, наприклад, CSRF захист, і надає більше контролю за доступом до даних. GET є зручним для таких речей, як форма пошуку, тому що URL, який представляє GET запит, можна легко зберегти в обране або відправити поштою.

Django дозволяє:

- підготувати дані для відображення у формі;
- створити HTML форми для даних;
- отримати та обробити надіслані формою дані.

Ви можете написати код, який все це буде робити, але Django може виконати більшу частину самостійно.

HTML `<form>` лише частина необхідної схеми. Серце всього механізму – клас `Form`. Як і модель в Django, яка описує структуру об'єкта, його поведінку і уявлення, `Form` описує форму, як вона працює і відображається для користувача.

В каталог `djangoapp` додайте файл `forms.py` з наступним змістом:

```
from django import forms
from djangoapp.models import Measurement

class MeasurementForm(forms.ModelForm):
    class Meta:
        model = Measurement
        fields = '__all__'
```

Таким чином ми створили Django форму для моделі `Measurement`, вибравши при цьому усі атрибути. Можна було вказати `['value', 'description']`, результат буде однаковий. Також у формах виконується валідація, можна додати власні умови — для цього потрібно перевизначити метод `clean`.

Для того, щоб форму можна було показати користувачу необхідно створити `View`, яке буде працювати із формою. Додамо наступний код до файлу `djangoapp/views.py`:

```
from django.urls import reverse_lazy
from djangoapp.forms import MeasurementForm

class MeasurementCreate(generic.CreateView):
    form_class = MeasurementForm
    template_name = 'create_measurement.html'
    success_url = reverse_lazy('measurement_list')
```

Зауважте, що всі імпорти повинні знаходитися на початку файлу. `CreateView` — клас, призначений для обробки запитів на

створення нових об'єктів бази даних. Для роботи з формою треба вказати `form_class` та назву шаблону.

Функція `reverse_lazy` дозволяє Django отримати url адресу за назвою, що вказана у змінній `name` URL шаблону, щоб уникнути написання повної адреси та помилок у разі внесення змін до адреси. `success_url` визначає url, на яку буде перенаправлений користувач у разі успішного створення об'єкту.

Додамо нову URL адресу для створення об'єкту `Measurement`:

```
from djangoapp.views import MeasurementCreate

urlpatterns = [
    ...
    url(r'^create/', MeasurementCreate.as_view(),
name='create_measurement')
]
```

Файл `create_measurement.html` може виглядати так:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Додати виміп</title>
</head>
<body>
    <form action="{% url 'create_measurement' %}"
method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <input type="submit" value="Submit" />
    </form>
</body>
</html>
```

До шаблону додається елемент `form` із атрибутом `action`, що визначає на яку адресу буде відправлений запит, та `method` — яким методом. У випадку створення нового об'єкту використовується запит типу `POST`. `{% url <url_name> %}` - це шаблонний тег,

що дозволяє отримати повну адресу за ім'ям цієї адреси у структурі URL Django.

Django забезпечує захистом проти Cross Site Request Forgeries. Для відправки форми через POST необхідно використовувати шаблонний тег `csrf_token`.

`form` є контекстною змінною, та містить поля разом із їхніми назвами. Метод `as_p` не є обов'язковим до застосування, він використовується лише для виведення полів форми з використанням тегів `<p></p>`.

Поле `input` з типом `submit` є фактично кнопкою, за допомогою якої користувач підтверджує відправлення даних до серверу.

Додайте посилання на нову сторінку до шаблону `measurement.html`:

```
<a href="{% url 'create_measurement' %}">Додати вимір</a>
```

Відкрийте у браузері головну сторінку застосування. Якщо ви додали посилання на сторінку, перейдіть за ним. Ви повинні побачити два поля для введення даних, спробуйте створити об'єкт. Після успішного додавання об'єкту, ви повинні побачити головну сторінку із списком усіх об'єктів, в тому числі і щойно доданий.

WSGI. Веб сервер приймає та обслуговує запити. Він може віддавати файли (HTML, зображення, CSS, і т.д.) безпосередньо зі своєї файлової системи. Однак він не може напряму працювати з Django застосуванням. Необхідний інтерфейс, який буде запускати Django застосування, передавати йому запити від веб клієнта, наприклад браузера, і повертати відповіді.

Для цього був розроблений Web Server Gateway Interface (WSGI) - стандарт взаємодії Python програм і веб-сервера. `uWSGI` є однією з реалізацій WSGI. Необхідно налаштувати `uWSGI` для створення Unix-сокету та взаємодії з веб-сервером за допомогою протоколу WSGI. Повний стек компонентів буде виглядати наступним чином:

веб-клієнт ⇔ веб-сервер ⇔ сокет ⇔ `uWSGI` ⇔ Django.

Встановити `uWSGI` для Python можна, якщо активувати віртуальне середовище, та виконати команду:

```
pip install uwsgi
```

Далі необхідно створити новий файл `uwsgi.ini` поруч із `manage.py` з наступним змістом:

```
[uwsgi]
chdir = /home/user/django-project/djangosite
module=djangosite.wsgi
socket = 127.0.0.1:8080
chmod-socket=666
home=/home/user/django-project/env
```

- `chdir` використовується для позначення каталогу проекту;
- `module` — Django wsgi файл;
- `socket` — вказує на адресу для підключення сокету. Якщо перейти за цією адресою у браузері, то користувач нічого не побачить, оскільки браузер використовує протокол `http`, а не `uWSGI`;
- `chmod-socket` — визначає права доступу до файлу сокета;
- `home` вказує на віртуальне оточення для вашого проекту.

Система Supervisor. Supervisor — це утиліта керування процесами в операційній системі. Нею можна скористатися, якщо є програми, які потребують перезапуску за певними правилами. Таким чином не потрібно буде писати підсистему управління (rc-скрипти, систему моніторингу та перезапуску) для таких програм.

Supervisor запускає процеси як свої підпроцеси, тому він має над ними повний контроль і знає їх точний стан.

`supervisorctl` надає системний контроль, а також веб-інтерфейси для моніторингу та керування процесами. Користувач може бачити стан програм та виконувати дії над ними (`start`, `stop`, `restart`).

Можна групувати програми і здійснювати над ними спільні дії (наприклад, перезавантаження всіх програм). Також є можливість вказувати пріоритет для кожної програми, тим самим організовуючи порядок перезапуску.

Ця утиліта написана на Python, працює на Linux, Mac OS X, Solaris і FreeBSD.

Необхідно додати запуск Django проекту до Supervisor. Якщо утиліта ще не встановлена, виконайте:

```
sudo apt-get install supervisor
```

Далі додайте файл `djangoproject.conf` до каталогу `/etc/supervisor/conf.d`:

```
[program:djangoproject]
directory=/home/user/django-project/djangosite
command=/home/user/django-project/env/bin/uwsgi --
ini uwsgi.ini
stdout_logfile=/home/user/django-
project/logs/supervisor.log
```

- `directory` вказує на головну директорію вашого проекту;
- `command` визначає безпосередньо команду яка буде виконуватися та відстежуватися;
- `stdout_logfile` вказує на файл, куди записується уся інформація щодо роботи вашої програми.

Перейдіть до каталогу `django-project` та виконайте:

```
mkdir logs
```

Виконайте команду для оновлення конфігурацій:

```
sudo supervisorctl reread
```

У терміналі повинні побачити наступне:

```
djangoproject: available
```

Далі потрібно виконати команду для перезапуску утиліти:

```
sudo supervisorctl reload
```

Виконайте команду:

```
sudo supervisorctl
```

Якщо все виконано правильно, ви побачите приблизно наступне:

```
django project                                RUNNING      pid
13390, uptime 0:00:07
```

Якщо замість цього и бачите помилку або статус FATAL, це означає, що конфігурація виконана неправильно. Переглянути деталі можна в файлі /home/user/django-project/logs.

NGINX – програмне забезпечення, написане для UNIX-систем. Основне призначення – самостійний HTTP-сервер, або, як його використовують частіше, фронтенд для високонавантажених проєктів. Можливе використання NGINX як SMTP/IMAP/POP3-сервера, а також зворотного TCP проксі-сервера.

Для встановлення:

```
sudo apt-get install nginx
```

Для запуску(start), зупинки(stop) або перегляду стану(status) nginx служби виконайте:

```
sudo service nginx <action>
```

Для того, щоб nginx застосував нові налаштування, виконайте:

```
sudo service nginx reload
```

Для перевірки правильності конфігурацій, існує команда:

```
sudo nginx -t
```

Перевірте статус програми, та запустіть якщо необхідно. До каталогу /etc/nginx/sites-available додайте файл django project:

```
upstream django {
    server 127.0.0.1:8080;
}

server {
```

```
listen      9000;
location / {
    uwsgi_pass  django;
    include     uwsgi_params;
}
}
```

В даному файлі конфігурації директива `upstream` описує сервер, що слухає на UNIX-сокеті локально за портом 8080 (збігає з параметром `socket` у файлі `uwsgi.ini`). Server задає адресу та інші параметри серверу. Директива `listen` відповідає за порт, за яким сайт буде доступним у браузері, `location` вказує куди повинен бути перенаправлений запит для оброблення. Таких директив може бути декілька, URL можуть бути задані регулярними виразами. В нашому випадку достатньо «/».

Перейдіть до `/etc/nginx/sites-enabled` та створіть посилання на щойно створений файл:

```
sudo ln -s /etc/nginx/sites-available/djangoproject
./djangoproject
```

Також у файлі `/etc/hosts` необхідно відредагувати рядок із локальною адресою та додати туди назву сайту за вашим бажанням, наприклад:

```
127.0.0.1      localhost djangoproject
```

Необхідно перезапустити `nginx` для застосування нових конфігурацій. Якщо налаштування були виконані правильно, то після переходу за адресою `http://djangoproject:9000/` у браузері повинен відобразитися ваш проект.

Порядок виконання лабораторної роботи

2.1 Застосувавши конспект лекцій та додаткову літературу, вивчити принципи створення проектів у Django.

2.2 Згідно з наведеним прикладом створити веб-застосування за індивідуальним завданням викладача.

Варіант 1. Створити базу даних лікарів та форми додавання та звільнення персоналу.

Варіант 2. Створити базу даних пацієнтів. Створити форми надходження та виписки пацієнтів.

Варіант 3. Створити базу даних медикаментів. Створити форми за призначанням та відділенням.

Варіант 4. Створити базу даних медичних інструментів. Створити форми за видом та призначенням інструменту.

Варіант 5. Створити графік чергувань лікарів. Створити форми відділення для чергування та заміни.

Варіант 6. Створити базу даних карток пацієнтів. Створити форми за лікарем та хворобою.

Варіант 7. Створити базу даних медичної техніки. Створити форми за призначанням та відділенням.

Варіант 8. Створити базу даних медикаментів. Створити форми закупівлі та використання медикаментів.

Варіант 9. Створити базу даних пацієнтів. Створити форми пацієнтів за лікарем та хворобою.

Варіант 10. Створити графік проведення операцій. Створити форми за лікарем та діагнозом.

2.3 Оформити звіт. Звіт з лабораторної роботи повинен містити: мету роботи; завдання на виконання роботи; тексти програм; результати роботи; висновки.

2.4 Відповісти на контрольні запитання.

Контрольні запитання

1. Перелічіть можливості Django.
2. Які головні принципи реалізовує Django?
3. Для чого використовується віртуальне середовище? Як його створити?
4. Дайте визначення WSGI.
5. Які проблеми вирішує Supervisor?
6. Наведіть загальний опис архітектури Django.
7. Як створити застосування та проєкт у Django? Чим вони відрізняються?

8. Які можливості існують у Django для роботи з БД?
9. Як створюються URL для сторінок веб-застосування у Django?
10. Що таке форма у Django та HTML?

ЛАБОРАТОРНА РОБОТА №2 ВИКОРИСТАННЯ AWS GREEN GRASS

Мета роботи

Ознайомитись з хмарними та граничними технологіями.

Теоретичні відомості

Під час цієї лабораторної роботи ви познайомитеся з AWS Greengrass, сервісом, який розширює можливості хмари Amazon Web Services на локальні пристрої. Greengrass дозволяє пристроям реагувати автономно під час використання захищеного зв'язку навіть без підключення до Інтернету. Після цієї вправи ви зрозумієте його базову архітектуру, вимоги та функціональність.

Лабораторне заняття полягає у створенні та конфігурації greengrass group. Для цього ви визначите ядро Greengrass (GGC) і два пристрої Greengrass, підключені до ядра (GGD). GGD спілкуватимуться між собою за допомогою топіків. Перед початком виконання вправи переконайтеся, що ви прочитали надану інформацію про AWS Greengrass.

Для виконання цієї вправи потрібен обліковий запис AWS, як-от безкоштовний пробний обліковий запис AWS, який є безкоштовним протягом одного року. Ви вже повинні були створити та запустити свій обліковий запис. Не всі облікові записи мають дозволи на використання Greengrass, як, наприклад, обліковий запис студентів. Тому переконайтеся, що ваш обліковий запис AWS дозволяє вам користуватися послугою.

Порядок виконання лабораторної роботи

Наступні кроки взято з офіційної документації Amazon Web Services і адаптовано до цього лабораторного заняття. Якщо у вас виникли проблеми під час їх впровадження, не соромтеся звернутися до офіційного [guide](#) AWS Greengrass.

1. Виконайте кроки, надані AWS, щоб налаштувати екземпляр EC2 для виділення AWS Greengrass [звідси](#). Після завершення цього

завдання (модуль 1) перегляньте цей документ ще раз, щоб ви могли побачити конфігурації для продовження.

2. Тепер ви встановите програмне забезпечення Greengrass Core [звідси](#). Переконайтеся, що ви назвали свою групу GG «GreengrassGroupxx», де «xx» — це номер вашої групи. При створенні групи ви завантажуйте дистрибутив ПЗ. Для примірника Amazon EC2 завантажте пакет для архітектури x86_64 і дистрибутива Linux. Після завершення цього завдання (модуль 2) зверніться до цього документу ще раз. Додайте скріншот вашої групи GG.

3. На цьому етапі ви повинні створити групу Greengrass і визначити її ядро Greengrass. Переконайтеся, що у вас є сертифікати та що служба Greengrass працює належним чином.

```
cd /greengrass/ggc/core/
sudo ./greengrassd початок
ps aux | grep PID-номер
```

4. У наступних кроках ви визначите пристрої Greengrass, які підключатимуться до ядра Greengrass, яке ви щойно створили. Обидва (пристрої та ядро) мають бути в одній групі greengrass. Щоб запустити приклади сценаріїв і AWS SDK, на вашому комп'ютері потрібно встановити Python.

5. Щоб створити пристрої, зверніться до модуля AWS: Взаємодія з пристроями в групі AWS IoT Greengrass ([тут](#)). Ви можете перевірити попередній модуль, якщо хочете, але це не обов'язково в цій лабораторній роботі.

ВАЖЛИВО: змініть назви пристроїв і тему, на яку вони підписані, щоб вказати номер вашої групи. Знову «xx» — номер вашої групи. **ЯКЩО ЦЕ НЕ ЗМІНИТИ, ВАШЕ НАДАННЯ БУДЕ НЕ ДІЙСНИМ.**

приклад:

Пристрій: PublisherGxx
 Пристрій: SubscriberGxx
 Тема: greengrass/groupxx

6. Тепер ви повинні змінити сценарій `basicDiscovery.py`, щоб опублікувати таке повідомлення:

```
{"Group": "Gtest", "sensorValues" : {"x":value[0], "y": value[1],  
"z": value[2]}}
```

Примітка. `<value[x]>` – це випадково згенероване число в діапазоні від 1 до 100.

Додайте знімок результату, який ви отримаєте після запуску сценарію. Він повинен показувати, як пристрій публікує значення.

ВАЖЛИВО: Кожен скріншот має відображати тему, що відповідає вашій групі.

7. Запустіть підписник на комп'ютері та прикріпіть скріншот результатів. Обробіть отримані дані, щоб показати наступний результат.

ВАЖЛИВО: Кожен скріншот має відображати тему, що відповідає вашій групі.

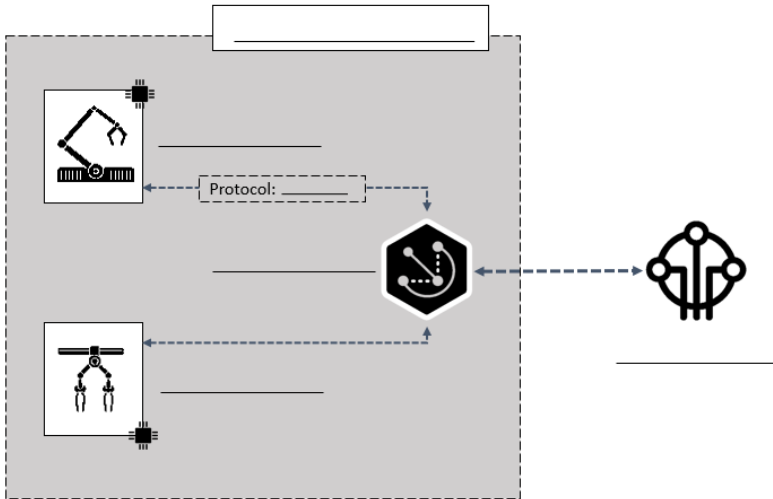
8. Дайте відповіді на наступні запитання щодо вправи, яку ви щойно виконали.

Контрольні запитання

Greengrass – це служба Edge чи хмарних обчислень? Коротко поясніть свою відповідь.

Скільки ядер може мати Greengrass group?

Заповніть порожні місця на малюнку, щоб описати базову архітектуру AWS Greengrass і пояснити кожен елемент.



Які типи протоколів використовує AWS Greengrass для захисту зв'язку між пристроями?

Що таке закритий і відкритий ключі? Коротко поясніть, як вони використовуються в системі.

Що станеться, якщо у вас немає цих сертифікатів (ключів) у папці, де знаходиться файл `basicDiscovery.py`?

Який сертифікат root-ca у вас є в папці пристроїв?

Які типи елементів можна вибрати з консолі AWS IoT, коли ви створюєте нову підписку в групі greengrass.

Які теми Greengrass?

Чи може пристрій виступати в якості видавця та передплатника в різних темах GG? Якщо так, то яким чином?

Чи можна передплатити кілька пристроїв на одного видавця? Якщо так, то яким чином?

Що таке служба відкриття?

Чому вам потрібно вказувати кінцеву точку aws

ЛАБОРАТОРНА РОБОТА №3 ПРОГРАМУВАННЯ ДЛЯ РІВНЯ ПРИСТРОЇВ З ВИКОРИСТАННЯМ PYTHON

Мета роботи

Навчитися працювати з платою розширення та датчиками для Raspberry Pi.

Теоретичні відомості

Часто Raspberry Pi використовують для роботи з низькорівневим обладнанням, зокрема, для отримання і аналізування даних, що надходять з різних датчиків. Для цього використовується шина загального введення-виведення GPIO (General Purpose Input Output). Для роботи з шиною розроблено безліч різних бібліотек.

Надалі, ці дані можна отримувати, підключаючись до Raspberry віддалено, або використовувати в інших програмах, що працюють на пристрої.

У даній лабораторній роботі ми скористаємося спеціально спроектованою навчальною платою розширення, яка оснащена, зокрема, датчиками освітленості і температури. Також на ній присутні 8 світлодіодів і CAN-шина для підключення різних пристроїв. Схема плати розширення представлена на рис. 3.1.

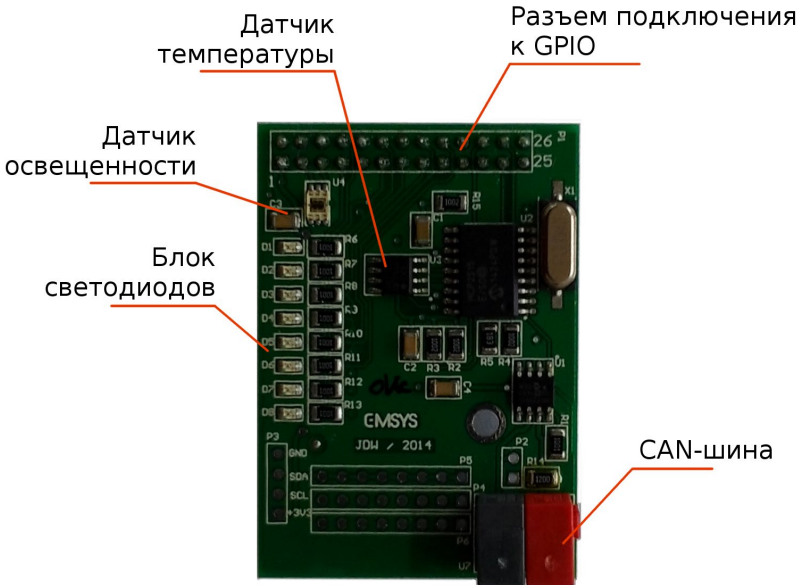


Рисунок 3.1 - Схема платы расширения для Raspberry Pi

Для роботи із платою за допомогою Python потрібно встановити бібліотеку **GPIO**, залежно від версії Python:

```
sudo apt-get update
sudo apt-get install python-rpi.gpio
sudo apt-get install python3-rpi.gpio
```

Основні команди модуля **RPi.GPIO**.

Імпортування модуля **RPi.GPIO**:

```
import RPi.GPIO as GPIO
```

Нумерація контактів:

```
GPIO.setmode(GPIO.BOARD)
# or
GPIO.setmode(GPIO.BCM)
```

Щоб визначити, яка система нумерації контактів була встановлена (наприклад, іншим модулем Python):

```
mode = GPIO.getmode()
```

Необхідно налаштувати кожен канал, який використовується, як вхідний або вихідний. Щоб налаштувати канал як вхідний:

```
GPIO.setup(channel, GPIO.IN)
```

Щоб налаштувати канал як вихідний:

```
GPIO.setup(channel, GPIO.OUT)
```

Для зчитування та встановлення стану контакту GPIO відповідно (state може бути 0 / GPIO.LOW / False або 1 / GPIO.HIGH / True):

```
GPIO.input(channel)
GPIO.output(channel, state)
```

Для очищення наприкінці скрипту, існує наступна команда:

```
GPIO.cleanup()
```

Нижче наведений приклад дозволяє виконувати маніпулювання світлодіодами:

```
import RPi.GPIO as GPIO
import time

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BOARD)
arr = [7, 8, 18, 16, 15, 13, 12, 11]

while True:
    for i in arr:
        GPIO.setup(i, GPIO.OUT)
        GPIO.output(i, True)
        time.sleep(0.1)
```



```

for i in arr:
    GPIO.setup(i, GPIO.OUT)
    GPIO.output(i, False)
    time.sleep(0.1)

```

Для маніпулювання світловими датчиками використаємо **SMBus** (System Management Bus) – протокол обміну даними, заснований на інтерфейсі I2C.

Встановимо `smbus-cffi` – Python модуль, що надає доступ до SMBus через інтерфейс I2C / dev на Linux:

```

sudo apt-get install python-pip
sudo apt-get install build-essential libi2c-dev
i2c-tools python-dev libffi-dev
pip install smbus-cffi

```

Основні команди модуля.

- `long write_quick(int addr)` - надіслати адресний байт;
- `long read_byte(int addr)` - прочитати один байт з пристрою, не вказуючи регістр пристрою;
- `long write_byte(int addr, char val)` - надіслати пристрою один байт;
- `long read_byte_data(int addr, char cmd)` - Read Byte Data транзакція читання байта;
- `long write_byte_data(int addr, char cmd, char val)` - Write Byte Data транзакція запису байта;
- `long read_word_data(int addr, char cmd)` - Read Word Data транзакція читання слова;
- `long write_word_data(int addr, char cmd, int val)` - Write Word Data транзакція запису слова;
- `long process_call(int addr, char cmd, int val)` - Process Call транзакція визову процесу;
- `long[] read_block_data(int addr, char cmd)` - Read Block Data транзакція читання блоку даних;
- `write_block_data(int addr, char cmd, long vals[])` – запис до 32 байтів; ця функція додає початковий байт, який вказує довжину масиву `vals` перед самим масивом - можна натомість використовувати `write_i2c_block_data`;

- `long[] block_process_call(int addr, char cmd, long vals[])` - Block Process Call транзакція, що поєднує у собі запис блоку з наступним читанням блоку за тією ж адресою пристрою.

Для перегляду приєднаних пристроїв можна виконати наступну команду:

```
sudo ls /dev/i2c*
```

Результатом буде список, наприклад, `/dev/i2c-0 /dev/i2c-2 /dev/i2c-4 /dev/i2c-6`.

Для того, щоб отримати доступ до пристрою у програмі, треба використати команду `smbus.SMBus()` та передати у якості аргумента останню цифру, що позначає девайс у вище наведеному списку, наприклад:

```
bus = smbus.SMBus(6)
```

Приклад програмної реалізації:

```
import smbus
import time
```

```
bus = smbus.SMBus(1)
address = 0x29
```

```
bus.write_byte(address, 0xa0)
bus.write_byte(address, 0x03)
time.sleep(0.5)
```

```
while True:
    bus.write_byte(address, 0xac)
    a = bus.read_byte(address)
    bus.write_byte(address, 0xad)
    b = bus.read_byte(address)
    print a + b*256
    time.sleep(0.1)
```

Порядок виконання лабораторної роботи

3.1 Застосувавши конспект лекцій та додаткову літературу, вивчити принципи програмування Raspberry Pi.

3.2 Використовуючи код прикладів, реалізувати програму за індивідуальним завданням.

Варіант 1. Використовуючи код прикладів, реалізувати програму, яка буде обробляти сигнали з Ph датчика і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 2. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру температури тіла і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 3. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру артеріального тиску (тонометр) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 4. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру пульсу і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 5. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру рівня кисню в крові (SPO2) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 6. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру повітряного потоку (дихання) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 7. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру ЕКГ (ECG) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 8. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру положення пацієнта (акселерометр) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 9. Реалізувати програму, яка буде обробляти сигнали з датчика м'язів / електроміографії (EMG) і поміщати отримані числа в заданий аргументом текстовий файл.

Варіант 10. Реалізувати програму, яка буде обробляти сигнали з датчика для виміру шкірно-гальванічної реакції (GSR - пітливість) і поміщати отримані числа в заданий аргументом текстовий файл.

3.4 Оформити звіт. Звіт з лабораторної роботи повинен містити: мету роботи; завдання на виконання роботи; тексти програм; результати роботи; висновки.

3.5 Відповісти на контрольні запитання.

Контрольні запитання

1. Що таке Raspberry Pi та чому її варто використовувати?
2. Які можливості надає Raspberry Pi?
3. Наведіть технічні характеристики Raspberry Pi.
4. Перелічіть ОС, що підтримуються Raspberry Pi.
5. Що потрібно для роботи з Raspberry Pi?
6. Що таке GPIO та яким чином можна нею керувати?
7. Дайте визначення та характеристику модулю SMBus.

ЛАБОРАТОРНА РОБОТА №4 ПРИНЦИПИ ТЕСТУВАННЯ У DJANGO

Мета роботи

Ознайомитися із принципами тестування у Django, навчитися створювати власні тести та аналізувати результати.

Теоретичні відомості

Автоматичні тести дають впевненість, що програма працює правильно. Такі тести можна запускати багаторазово. Їхнє успішне виконання покаже розробнику, що його зміни не призвели до порушення роботи програми. Провалений тест дозволить виявити, що в коді були зроблені зміни, які суттєво змінюють або ламають його поведінку. Дослідження помилки, яку видає провалений тест, і порівняння очікуваного результату з отриманим дасть можливість зрозуміти, де виникла помилка, в коді або у вимогах.

Все вищесказане справедливо для будь-яких тестів. Відмінністю ж модульного (юніт) тестування є те, що тестується не вся система в цілому, а невеликі її частини. Тобто тестування проводиться з високою гранулярністю. Це основна відмінність юніт-тестів від системних, коли тестування зазнає вся система чи підсистема, а також від інтеграційних, які перевіряють взаємодію між модулями.

Основна перевага незалежного тестування маленької ділянки коду полягає в тому, що якщо тест провалиться, помилку буде легко виявити і виправити. «Юніт» можна визначити як маленьку, зв'язну ділянку коду. Це цілком узгоджується з основним принципом розробки і часто юніт — це певний клас. Але це також може бути набір функцій або кілька маленьких класів, якщо весь функціонал неможливо розмістити в одному. Важливо пам'ятати, якщо ви жорстко запрограмуєте залежності досліджуваного класу від інших класів, помилку, яка викликала падіння тесту, буде складно локалізувати, і висока гранулярність, яка необхідна для юніт-тестів, буде втрачена. Тобто для написання юніт-тестів необхідна відсутність зчеплення.

Тестування у Django. Тестування відіграє життєво важливу роль у процесі створення якісного програмного забезпечення. Чим складніше сайт, тим більше часу потрібно на його перевірку і налагодження. На жаль, існує безліч прикладів, коли розробники і замовники оминають етап тестування сайту, що практично завжди призводить до великих фінансових і тимчасових витрат надалі, невдоволення користувачів ресурсу, і, в результаті, необхідності доопрацювання (або навіть повторної розробки) ресурсу.

Тестування веб-застосунів є складною задачею тому, що веб-застосування складається з декількох шарів логіки – від HTTP запитів на рівні обробки запитів до форм перевірки і рендеринга сторінок. За допомогою тестового фреймворку, вбудованого в Django, і декількох утиліт ви можете емулювати запити користувача і перевіряти яка сторінка з'явилася у відповідь. Для створення тестів краще використовувати стандартний модуль Python unittest. Ви можете використовувати і інші фреймворки для тестування, в Django є API і утиліти для швидкої інтеграції.

Створення і запуск тестів. Юніт-тести для Django використовують стандартну бібліотеку Python: unittest. Ця бібліотека дозволяє створювати тести в ООП стилі. У прикладі нижче відбувається наслідування від `django.test.TestCase`, який є класом-спадкоємцем `unittest.TestCase`, він запускає кожний тест в окремій транзакції для їхньої ізоляції.

Коли ви запускаєте тести, відбувається пошук всіх тестів (класів успадкованих від `unittest.TestCase`) в файлах, які починаються з `test`, потім створюється набір тестів (`test suite`) та їх запуск.

Шаблон програми, який використовує команда `startapp`, містить файл `tests.py`. Це підходить для додатків, які містять не багато тестів. Але із зростанням кількості тестів виникає необхідність розділити їх на модулі, наприклад `test_models.py`, `test_views.py`, `test_forms.py` і т. д. Ви можете використовувати будь-яку зручну вам структуру.

Основні будівельні блоки юніт-тестів — одиночні сценарії, які повинні бути налаштовані і перевірені на коректне виконання. У unittest тестові випадки представлені екземплярами класу `TestCase`. Для того, щоб зробити свій тестовий випадок, ви повинні написати підклас `TestCase`. Тестовий код примірника `TestCase` повинен бути

повністю самодостатнім, так щоб його можна було запускати в ізоляції або в довільній комбінації з будь-якою іншою кількістю тестових випадків.

Зверніть увагу, що для того, щоб щось перевірити, використовується методи `assert*()` або `fail*()`, які передбачені базовим класом. Якщо тест завершується невдачею під час виконання тесту, то буде викликано виняток, і тест буде визначений як провалений.

Щоб запустити тести, використовуйте команду `test` скрипта `manage.py` у вашому проєкті:

```
./manage.py test
```

Також можна передати шлях до каталогу, в якому буде виконаний пошук тестів:

```
./manage.py test project/
```

Можна вказати шаблон для пошуку файлів з тестами, використовуючи опцію `-p` (або `--pattern`), якщо імена файлів з тестами не відповідають шаблону `test*.py`:

```
./manage.py test -pattern="tests_*.py"
```

Якщо натиснути `Ctrl-C` при виконанні тестів, буде завершено поточний тест і потім зупинено виконання тестів. Після зупинки буде виведений результат виконання тестів і видалена тестова база даних. Комбінація `Ctrl-C` може бути дуже корисною, якщо ви забули вказати опцію `--failfast`, і не хочете чекати закінчення виконання всіх тестів, якщо один з них вже виконався з помилкою.

Наведений приклад дозволяє протестувати веб-застосування:

```
from          django.core.urlresolvers          import
reverse_lazy
from django.test import TestCase
from djangoapp import models

class TestCalls(TestCase):
    def setUp(self):
```

```

        self.create_url = 'create_measurement'
        self.list_url = 'measurement_list'
    def test_call_view_loads(self):
        for url in [self.create_url,
self.list_url]:
            path = str(reverse_lazy(url))
            response = self.client.get(path)

self.assertEqual(response.status_code, 200)
            self.assertTemplateUsed(response,
'{}.html'.format(url))
        def test_call_view_fails_blank(self):
            path =
str(reverse_lazy(self.create_url))
            response = self.client.post(path, {})
            self.assertFormError(response, 'form',
'value',
                                u'This field is
required.')
```

```

        def test_call_view_fails_incorrect(self):
            path =
str(reverse_lazy(self.create_url))
            value = '1'*51
            measurement_params = {
                'value': value,
            }
            response = self.client.post(path,
measurement_params)
            self.assertFormError(response, 'form',
'value', u'Ensure this value has at most 50 characters
(it has 51).')
```

```

        def test_call_view_succeed(self):
            path =
str(reverse_lazy(self.create_url))
            value = '1'*10
            description = 'measurement description'
            measurement_params = {
                'value': value,
                'description': description
            }
            response = self.client.post(path,
measurement_params)
```



```

        self.assertEqual(response.status_code,
302)
        measurement =
models.Measurement.objects.get(id=1)
        for field in
measurement_params.iterkeys():
            measurement_field =
getattr(measurement, field)
            self.assertEqual(measurement_field,
measurement_params[field])

```

В даному прикладі створюється екземпляр `django.test.TestCase`. Функція `setUp` містить дії, що будуть виконані до початку тестування.

Функція `test_call_view_loads` відповідає за тестування клієнтської частини: за посиланнями відправляються GET запити, перевіряється статус відповіді та шаблон, що був використаний.

У функції `test_call_view_fails_blank` відбувається перевірка поведінки форми за умови надсилання до неї незаповнених даних, відповідь повинна містити помилку.

Функція `test_call_view_fails_incorrect` містить перевірку поведінки форми за умови надсилання некоректних даних (у даному випадку, значення параметру `value` не повинно перевищувати 50 символів, що зазначено у моделі даних).

У функції `test_call_view_succeed` міститься перевірка правильності створення об'єкту.

У даному випадку запуск тестів можна здійснити за допомогою команди:

```
./manage.py test
```

Результатом виконання тестів повинне бути наступне:

```
Creating test database for alias 'default'...
```

```
-----
```

```
Ran 4 tests in 0.037s
```

```
OK
```

Якщо ви отримали результат FAILED, це означає, що функціональність застосування порушена та містить помилки або помилка була допущена при написанні тестів. Необхідно проаналізувати помилку та визначити причину її появи.

Порядок виконання лабораторної роботи

4.1 Застосувавши конспект лекцій та додаткову літературу, вивчити принципи тестування у Django.

4.2 Протестувати роботу веб-застосування та виявити чи має застосування порушення у функціональності або містить помилки, а також виявити та виправити помилки, якщо вони були допущені при написанні тестів.

4.3 Оформити звіт. Звіт з лабораторної роботи повинен містити: мету роботи; завдання на виконання роботи; тексти програм; результати роботи; висновки.

4.4 Відповісти на контрольні запитання.

Контрольні запитання

1. Що таке модульне тестування та у чому його відмінність?
2. Наведіть переваги юніт-тестування.
3. Що таке TDD?
4. Яким чином виконується тестування у Django?
5. Як організований тестовий код?
6. Як можна виконати запуск тестів? Чи можна перевірити лише один тестовий клас (case) або один тестовий метод?
7. Яка БД використовується для тестових даних у Django?
8. Чи можна змінити рівень детальності виведення результатів тестування?

ЛАБОРАТОРНА РОБОТА № 5. РОЗРОБЛЕННЯ ПІДСИСТЕМ АНАЛІЗУ ДАНИХ

Мета роботи

Мета роботи навчитись преобразовувати данні з знання за допомогою бібліотек python

Теоретичні відомості

Python є високорівневою мовою програмування загального призначення, яка орієнтована на підвищення продуктивності роботи розробника і читабельності коду. Python підтримує декілька парадигм програмування, в тому числі структурне, об'єктно-орієнтоване, функціональне, імперативне і аспектно-орієнтоване. Основні архітектурні риси — динамічна типізація, автоматичне керування пам'яттю, повна інтроспекція, механізм обробки винятків, підтримка багатопоточних обчислень і зручні високорівневі структури даних. Код на Python організовується у функції та класи, які можуть об'єднуватися в модулі (вони в свою чергу можуть бути об'єднані в пакети).

Основні переваги Python:

- швидкість виконання програм, написаних на Python дуже висока: це пов'язано з тим, що основні бібліотеки Python написані на C++ і виконання завдань займає менше часу, ніж на іншими мовами високого рівня;

- також існує можливість створення власних модулів для Python на C або C++;

- мова характеризується чітким і послідовним синтаксисом, продуманою модульністю і масштабованістю, завдяки чому початковий код написаних на Python програм можна легко прочитати;

- у стандартних бібліотеках Python можна знайти засоби для роботи з електронною поштою, протоколами Інтернету, FTP, HTTP, бази даних, тощо;

- скрипти, написані за допомогою Python виконуються на більшості сучасних ОС, така переносимість забезпечує Python популярність в різних областях програмування

Порядок виконання лабораторної роботи

1. Вивчити теоретичний матеріал, щодо бібліотек numpy, pandas та scikitlearn
2. Розширити проект з використанням бібліотек аналізу даних, відповідно до індивідуального завдання.

Контрольні запитання

ЛАБОРАТОРНА РОБОТА №6. ВЗАЄМОДІЯ З БАЗАМИ ДАНИХ

Мета роботи

Мета роботи навчитись працювати з нереляційними хмарними базами даних.

Теоретичні відомості

Існує два основних типи баз даних - реляційні та нереляційні бази даних.

Реляційні бази даних дозволяють нам зберігати, отримувати доступ та маніпулювати даними стосовно іншого фрагменту даних у базі даних. Дані зберігаються в організованих таблицях з рядками та стовпцями зі зв'язками, що зв'язують інформацію між таблицями. Для роботи з цими базами даних ми використовуємо мову структурованих запитів (SQL), наприклад, MySQL і PostgreSQL.

Нереляційні бази даних зберігають дані в реляційної, ні в табличній формі, як і реляційних базах даних. Вони також називаються базами даних NoSQL, тому що ми не використовуємо SQL для взаємодії з ними.

MongoDB і коли його використовувати

MongoDB - це сховище документів та нереляційна база даних. Це дозволяє зберігати дані в колекціях, які складені з документів.

У MongoDB документ є просто JSON-подібний двійковий формат серіалізації, званий BSON або Binary-JSON, і має максимальний розмір 16 мегабайт. Це обмеження розміру використовується для забезпечення ефективного використання пам'яті та смуги пропускання під час передачі.

MongoDB також надає специфікацію GridFS у разі потреби зберігати файли, розмір яких перевищує встановлене обмеження.

Документи складаються з пар ключ-значення, як у звичайних даних JSON. Однак цей формат BSON також може містити більше типів даних, таких як Date та Binary Data. BSON був розроблений, щоб бути легким та ефективним при кодуванні та декодуванні даних у та з BSON.

Будучи сховищем даних NoSQL, MongoDB дозволяє нам скористатися перевагами використання нереляційної бази даних у

порівнянні з реляційною. Однією з переваг є те, що забезпечує високу масштабованість за рахунок ефективного масштабування по горизонталі за рахунок поділу даних і розміщення їх на декількох комп'ютерах.

MongoDB також дозволяє нам зберігати великі обсяги структурованих, напівструктурованих та неструктурованих даних без необхідності підтримувати відносини між ними.

Як і в будь-якого іншого рішення, MongoDB має свої недоліки. Перший у тому, що не підтримує відносини між збереженими даними. Через це важко виконувати транзакції ACID, які забезпечують узгодженість.

Інтеграція Python з MongoDB

MongoDB надає драйвери та інструменти для взаємодії зі сховищем даних MongoDB з використанням різних мов програмування, включаючи Python, JavaScript, Java, Go та C#.

PyMongo є офіційним драйвером MongoDB для Python, і ми будемо використовувати його для створення простого скрипту, який ми використовуватимемо для маніпулювання даними, що зберігаються в нашій базі даних SeriesDB.

З встановленими Python 3.6 + та Virtualenv, давайте створимо віртуальне середовище для програми та встановити PyMongo за допомогою pip:

```
virtualenv --python=python3 env --no-site-packages
source env/bin/activate
pip install pymongo
```

Підключення к MongoDB

Спочатку ми імпортуємо pymongo в mongo_db_script.py і створюємо клієнт, підключений до нашого локально працюючого екземпляра MongoDB:

```
import pymongo
```

```
# Create the client
```

```
client = MongoClient('localhost', 27017)
```

```
# Connect to our database
```

```
db = client['SeriesDB']
```

```
# Fetch our series collection
```

```
series_collection = db['series']
```

Ми створили клієнт, який підключається до нашого сервера MongoDB, та використовували його для отримання нашої бази даних 'SeriesDB'. Потім ми отримуємо нашу колекцію 'series' і зберігаємо її у змінній.

Створення документів

Щоб зробити наш сценарій зручнішим, ми напишемо функції, які дозволяють нам легко маніпулювати даними з PyMongo. Ми будемо використовувати Python словники для подання документів і передамо ці словники нашим функціям. По-перше, давайте створимо функцію для вставки даних у нашу колекцію 'series':

```
# Imports truncated for brevity
```

```
def insert_document(collection, data):
    """ Function to insert a document into a collection and
    return the document's id.
    """
```

```
    return collection.insert_one(data).inserted_id
```

Ця функція отримує колекцію та словник даних та вставляє дані до наданої колекції. Потім функція повертає ідентифікатор, який можна використовувати для точного запиту окремого об'єкта з бази даних.

Слід також зазначити, що MongoDB додає додатковий ключ `_id` до наших документів, коли вони не надаються при створенні даних.

Тепер давайте спробуємо додати шоу, використовуючи нашу функцію:

```
new_show = {
    "name": "FRIENDS",
    "year": 1994
}
print(insert_document(series_collection, new_show))
```

В консолі получимо:

```
5e4465cfdcbdbdc68a6df233f
```

Коли ми запускаємо наш скрипт, `_id` нового запису друкується в терміналі, і ми можемо використовувати цей ідентифікатор для отримання запису пізніше.

Ми можемо надати значення `_id` замість того, щоб призначати його автоматично, що ми надали б у словнику:

```
new_show = {
    "_id": "1",
    "name": "FRIENDS",
    "year": 1994
}
```

І якби ми спробували зберегти документ з існуючим `_id`, нас зустріне помилка, подібна до наступної:

```
DuplicateKeyError: E11000 duplicate key error index:
SeriesDB.series.$id dup key: { : 1 }
```

Отримання документів

Для вилучення документів з бази даних ми будемо використовувати `find_document()`, яка просить нашу колекцію для одного або кількох документів. Наша функція отримає словник, який містить елементи, за якими ми хочемо фільтрувати, та необов'язковий аргумент, щоб вказати, чи хочемо ми один документ чи кілька документів:

Imports and previous code truncated for brevity

```
def find_document(collection, elements, multiple=False):
    """ Function to retrieve single or multiple documents from a
    provided
```

```
    Collection using a dictionary containing a document's elements.
    """
```

```
    if multiple:
```

```
        results = collection.find(elements)
```

```
        return [r for r in results]
```

```
    else:
```

```
        return collection.find_one(elements)
```

А тепер давайте використовувати цю функцію, щоб знайти деякі документи:

```
result = find_document(series_collection, {'name': 'FRIENDS'})
print(result)
```

При виконанні нашої функції ми не надали параметра `multiple`, і в результаті ми отримали один документ:

```
{'_id': ObjectId('5e3031440597a8b07d2f4111'), 'name': 'FRIENDS',
'year': 1994}
```


Коли параметр `multiple` надано, результатом є список усіх документів у нашій колекції, для яких встановлено атрибут `name` як `FRIENDS`.

Оновлення документів

Наша наступна функція `update_document()` буде використовуватися для оновлення одного конкретного документа. Ми будемо використовувати `_id` документа та колекцію, до якої він належить, при його пошуку:

Imports and previous code truncated for brevity

```
def update_document(collection, query_elements, new_values):
    """ Function to update a single document in a collection.
    """
```

```
    collection.update_one(query_elements, {'$set': new_values})
```

Тепер давайте вставимо документ:

```
new_show = {
    "name": "FRIENDS",
    "year": 1995
}
```

```
id_ = insert_document(series_collection, new_show)
```

Зробивши це, давайте оновимо документ, використовуючи повернутий `_id`:

```
update_document(series_collection, {'_id': id_}, {'name':
'F.R.I.E.N.D.S'})
```

І, нарешті, давайте подивимося його, щоб переконатися, що нове значення було встановлено, і надрукуємо результат:

```
result = find_document(series_collection, {'_id': id_})
print(result)
```

Коли ми виконуємо наш скрипт, побачимо, що наш документ оновлено:

```
{'_id': ObjectId('5e30378e96729abc101e3997'), 'name':
'F.R.I.E.N.D.S', 'year': 1995}
```

Видалення документів

І, нарешті, давайте напишемо функцію для видалення документів:

Imports and previous code truncated for brevity

```
def delete_document(collection, query):
```

```
""" Function to delete a single document from a collection.
"""
```

```
collection.delete_one(query)
```

Оскільки ми використовуємо метод `delete_one`, лише один документ можна видалити за виклик, навіть якщо запит відповідає кільком документам.

Тепер давайте використовуємо функцію для видалення запису:

```
delete_document(series_collection, {'_id': id_})
```

Якщо ми спробуємо отримати той самий документ:

```
result = find_document(series_collection, {'_id': id_})
```

```
print(result)
```

Результатом буде:

```
None
```

Наступні кроки

Усі доступні методи можна знайти в офіційній документації PyMongo та класифікувати відповідно до підмодулів.

Порядок виконання лабораторної роботи

1 Застосувавши конспект лекцій та додаткову літературу, вивчити принципи використання NoSQL баз даних та методи взаємодії з MongoDB.

2 Додати до розробленого у другій роботі вебзастосування базу даних MongoDB та виконати основні команди для взаємодії з нею.

3 Оформити звіт. Звіт з лабораторної роботи повинен містити: мету роботи; завдання на виконання роботи; тексти програм; результати роботи; висновки.

4 Відповісти на контрольні запитання.

Контрольні запитання

1. Які типи баз даних ви знаєте?
2. Наведіть переваги та недоліки NoSQL баз даних.
3. Як підключити MongoDB до проєкту на Python?
4. Які основні команди для взаємодії з MongoDB ви знаєте?

ЛІТЕРАТУРА

1. Петин В.А. Микрокомпьютеры Raspberry Pi. Практическое руководство. – СПб.: БХВ-Петербург, 2015. – 240 с.
2. Мацяшек Л.А. Анализ требований и проектирование систем. Разработка информационных систем с использованием UML / Л.А. Мацяшек // М.:Изд. дом «Вильямс», 2002. – 432 с.
3. Описание Rational Software Architect for WebSphere Software версии 7.5: [Электрон. ресурс]. – Режим доступа: http://www.ibm.com/developerworks/ru/library/r-0926_arnold/index.html.
4. Что нового в IBM Rational Software Architect 8.0: [Электрон. ресурс]. – Режим доступа: <http://www.ibm.com/developerworks/ru/library/r-whats-new-in-rational-software-architect-8/>.
5. Введение в IBM Rational Software Architect: [Электрон. ресурс]. – Режим доступа: <http://www.ibm.com/developerworks/ru/library/kunal/>.
6. Кватрани, Ю. Визуальное моделирование с помощью Rational Rose 2002 и UML: пер. с англ. [Текст] / Кватрани Ю. – М. : Вильямс. 2002. – 240 с.
7. Кватрани, Т. Визуальное моделирование с помощью IBM Rational Software Architect и UML: пер. с англ. [Текст] / Т. Кватрани. Дж. Палистрант. – М.: КУДИЦ-ПРЕСС. – 2007. – 192 с.
8. Ларман, К. Применение UML и шаблонов проектирования. 2-е издание [Текст] / Ларман К. – М. : Вильямс, 2002. – 624 с.
9. Соммервилл, И. Инженерия программного обеспечения: пер. с англ. [Текст] / Соммервилл И. – М.: Вильямс. 2002. – 624 с.
10. Шаблоны [Patterns] проектирования: [Электронный ресурс]. – Режим доступа: <http://oad.asf.ru/Patterns.aspx>.
11. Шаллоуей А. Шаблоны проектирования. Новый подход к объектно-ориентированному анализу и проектированию. Пер. с англ. – М.: Изд. дом «Вильямс», 2002. – 288 с.